

Build Plan: Demand Forecasting & Prescriptive Inventory Management Dashboard

USTore Capstone Project 2 (IS 26316) — a phased plan for building the system bit by bit

Two Principles to Lock First

1. The historical data is your critical path — not the live interface

Chapter 4's results run mostly on the historical 2023–2026 tally records, which your manuscript already names as the initial training baseline. So the path to a defensible system is: historical data → ETL → star schema → FSN → Prophet → prescriptive math → dashboard. Build and deploy the Digital Tallying Interface in parallel, but never let it block the analytics.

2. One SQLite database is the single source of truth

The interface writes to it (live), the ETL bulk-loads history into it, the Python analytics reads from it and writes its results (FSN classes, forecasts, ROP/EOQ) back into it as result tables, and Power BI reads from it. Power BI does NOT run Prophet — Prophet runs in Python as a batch job, and Power BI only displays the numbers Python already computed.

Decisions to Lock in Phase 0

Your manuscript deliberately left two choices open. Recommended path:

- **Interface + warehouse:** Flask + SQLite. One Python ecosystem, clean schema enforcement, one file for Power BI to connect to. Keep “Google Form → Sheets → import” only as a fallback if you cannot host the Flask app on campus.
- **Why not React / Google Sheets as primary:** React adds front-end overhead you don't need for an internal tally form; Sheets weakens schema validation and splits your source of truth. Staff adoption is not your Chapter 4 blocker, so optimise for a clean, demonstrable build.
- Collect raw inputs now: all monthly Excel tally files (2023–2026), the May 2024 Daily Sales Report, monthly beginning-inventory count sheets, and the UST academic calendar.
- Gather the verbal estimates the math needs: supplier lead time, ordering cost, holding cost. Store them in `Dim_Parameters` so they stay editable.

The Phased Build

PHASE 0

Setup & Decisions (~1 week)

- Lock the stack (see recommendation above); set up a shared Git repo and a pinned Python environment (pandas, prophet, sqlite).
- Assign owners for each workstream (see division of labour below).
- Assemble all raw data files and the academic calendar in one shared folder.

PHASE 1

Data Foundation: Schema + ETL (the foundation everything depends on)

The biggest block. Build carefully — every downstream result inherits its quality.

- Write DDL for all tables: Fact_Sales, Dim_Product, Dim_Date, plus Dim_Parameters and Event_Log. (Dim_Inventory stays omitted — stock is derived.)
- Build and populate Dim_Date from the academic calendar: set every boolean flag (is_enrollment_period, is_exam_week, is_event_day, is_sem_break, is_store_closed, is_tally_date) and semester_week.
- **Build the controlled vocabulary mapping file for item names first.** This is the make-or-break task — your own “UST College ID Lace / GENERIC Lace” example shows why. Do it manually with USTore input before writing transform code.
- Build the ETL (pandas): wide→long pivot, apply vocabulary mapping, proportional imputation for price-grouped rows (tag imputation_flag, weight 0.5), normalise dates to ISO 8601, derive daily_depletion_rate and cumulative_monthly_units, and route unresolvable rows to an exception table instead of deleting them.
- Bulk-load the transformed history into SQLite.

■ Ch 4 output: *percentage of records flagged / isolated, reported as a data quality metric.*

PHASE 2

FSN Classification

- Compute ADUS per SKU (active tally dates as denominator for history, calendar days for live data); apply the 80th-percentile cutoff.
- Detect High-Velocity Limited (HVL) items and apply the Stock Depletion Rate rule (>80% of stock sold in the first 14 days → Fast).
- Run the threshold sensitivity analysis at 75 / 80 / 85; flag category-switching SKUs as borderline. Write fsn_class back to Dim_Product.

■ Ch 4 output: *category counts, borderline-item list, and the sensitivity table.*

PHASE 3

Prophet Forecasting (Fast + HVL SKUs only)

- Implement the data-sufficiency tiers: 60+ obs → standard fit with 80/20 split; 30–59 → simplified model (reduced changepoints, fixed linear trend) + LOOCV on the last 20; <30 → no Prophet, assign a 30-day rolling average and flag “Insufficient Data for Forecasting”.
- Fit with academic-calendar regressors; tune lower_window / upper_window; MCMC at 1,000 samples.
- Validate: walk-forward; MAPE ≤ 20% for standard periods; MAE as the primary metric for semestral breaks; RMSE as a diagnostic; compare against the naive baseline (previous-period demand); run the split-period evaluation (historical-tally vs live-interface periods).

■ Ch 4 output: *per-SKU metric table, observation counts/tiers, and the naive-vs-Prophet comparison.*

PHASE 4

Prescriptive Math: ROP / Safety Stock / EOQ

- Implement the three formulas with FSN-differentiated z-scores (1.65 fast = 95% service level, 1.04 slow = 85%, non-moving excluded) and the coefficient-of-variation fallback for sparse standard deviation.
- Run the EOQ sensitivity check: compare total cost at 0.5x, 1x, and 2x EOQ per SKU.
- Write ROP / Safety Stock / EOQ into a summary results table.

■ Ch 4 output: *per-SKU prescriptive table and the EOQ sensitivity results.*

PHASE 5**Digital Tallying Interface (parallel — starts once the schema exists)**

- Flask form logging item / quantity / date / supplier / transaction_type, with field-level validation (non-null enforcement).
- Event flagging → writes Event_Log; store-closure toggle → sets is_store_closed; intra-day completeness check → “Data Pending” alert and the “Unverified Zero” flag.
- Incremental daily append into Fact_Sales.

PHASE 6**Power BI Dashboard**

- Connect to the SQLite DB / exported result tables.
- Build the five views in narrative order: Stock Status → FSN Classification → Demand Forecast → Restocking Advisory → Automated Batch Sales Report (with PDF export).
- Add the calendar-contextual interpretation cards.

PHASE 7**Chapter 4 Write-up**

- Assemble every metric and table above into Results, Data Analysis, and the dashboard walkthrough sections.

Suggested Division of Labour (4 members)

Member	Owns	Phases
A	ETL + star schema — the foundation everyone depends on	0–1
B	Flask tallying interface (starts once the schema exists)	5
C	FSN classification + prescriptive math	2, 4
D	Prophet forecasting + validation (most model-heavy)	3
All	Power BI dashboard, then each member's own Ch 4 sections	6–7

Formula Reference (plain-text form)

ROP	ROP = (Average Daily Demand x Lead Time in Days) + Safety Stock
Safety Stock	Safety Stock = Z x sigma_demand x sqrt(Lead Time)
EOQ	EOQ = sqrt((2 x D x S) / H)

Where D = annualised 30-day Prophet forecast, S = ordering cost per cycle, H = holding cost per unit per year. Average Daily Demand comes from the Prophet 30-day point forecast; Lead Time from Dim_Product.

Four Gotchas Specific to Your Project

- The vocabulary mapping in Phase 1 will take longer than expected, and everything downstream inherits its errors. Front-load it.
- Only a subset of core apparel items will qualify for real Prophet models; most SKUs land in the rolling-average fallback. That is not a failure — it is in your manuscript, and it is honest.
- Do not forecast inside Power BI. Batch in Python, display in Power BI.
- Fix a single “analysis snapshot date” for the defence so all your numbers are reproducible.

Build Order at a Glance

[Phase 0] → [Phase 1: ETL + schema] → [Phase 2: FSN] → [Phase 3: Prophet] → [Phase 4: ROP/EOQ] → [Phase 6: Dashboard] → [Phase 7: Ch 4]

Phase 5 (the tallying interface) runs in parallel from the moment the Phase 1 schema exists, and feeds live data into the same database without holding up the analytics chain above.

Recommended first brick: the full SQLite schema DDL for all five tables, or the Dim_Date builder, or the ETL wide→long pivot. Each is self-contained and unblocks the next phase.